NovaCRM Customer Onboarding Automation: Requirements, Workflow Analysis and Design

Author: Praveen Kumar Singh · Forward Deployed Engineer take-home for Rocketlane · Draft 5, 27 Oct 2026

1. Summary

NovaCRM's CS team turns every closed deal into an onboarding project by hand. That costs time, and three things go wrong repeatedly: the wrong template gets used (Enterprise vs Growth), late tasks are not escalated, and data-migration tasks are closed without the customer's data being verified.

This system automates the first mile of onboarding and leaves judgement to people where it matters:

1. An **Intake & Routing Agent** watches the CS inbox, reads the AE's deal email, checks that every required field is present (and asks the AE when one is missing), **phones the AE** to confirm the plan tier, and creates the Rocketlane project from the matching template.
2. A **Communication Agent** creates the customer's Slack channel with a name, topic and welcome message that fit the tier and the customer.
3. A **Rocketlane automation** (configured in Rocketlane, not code) escalates overdue tasks: to the Project Manager at 1 day overdue, to the Project Owner at 4 days overdue.

The design rule throughout: **the language model reads and writes text; plain code decides.** The model extracts fields from free-text email, interprets what the AE said on the call, and writes one personal line of the welcome message. Every decision that changes something (which template, whether to proceed, whether to retry or escalate) is made by deterministic, tested code, and every step is written to an audit log with its reason.

2. What changes for your team

Who	Today	With this system
AE	Writes an email, then answers CS questions in DMs	Writes the same email. Gets a short automated call: "Enterprise or Growth?" If something is missing, gets one reply in the same thread saying exactly what
CSM	Builds the project and the Slack channel by hand	Finds the project (right template, their name on it) and the customer channel ready within minutes of the email
Priya (CS Manager)	Chases late tasks from memory; finds template mix-ups late	Gets an automatic notice when a task is 4 days overdue, and a message with the reason whenever the system cannot decide on its own. One command settles it
Customer	Waits for someone to set things up	Gets a Rocketlane invitation and a Slack welcome on day one. Signs off their own migrated data before go-live

What still needs a person: a tier the AE cannot confirm, a project that does not match its template, and every escalation decision after the notice.

3. The current workflow (as-is)

#	Step	Who	Tool	What goes wrong today
1	Deal closes; AE emails the CS inbox with the customer name and the Salesforce opportunity link	AE	Email	Free text; details vary by AE; no plan tier
2	CS reads the email and works out the tier	CS team	Email, Salesforce	Manual lookup; sometimes guessed
3	Create the project from the onboarding template (≈15 tasks, 4 phases)	CS team	Asana	**Templates get mixed up**
4	Create a shared Slack channel with the customer	CS team	Slack	Manual; inconsistent naming and first message
5	Schedule the kickoff call	CSM	Calendar	Manual
6	Work through each phase's checklist	CSM	Asana	Status depends on people remembering
7	Escalate any task blocked > 3 days	CSM / CS Manager	none	**Falls through the cracks**
8	Data migration tasks marked done	CSM	Asana	**Done without the customer verifying the data**
9	Handoff summary; move to Support	CSM	Docs	Manual

ID	Pain	Root cause	Addressed by
P1	Wrong template	A person infers the tier under time pressure	The AE confirms the tier on a call; code maps tier → template; the created project is checked (D4)
P2	Late tasks not escalated	Escalation relies on memory	Rocketlane automation at +1 day (PM) and +4 days (Owner) (§10)
P3	Data marked migrated but not verified	"Done" means the CSM did their part	A sign-off task owned by the customer, which go-live depends on, in both templates (§9)
P4	Delay from close to kickoff	Every step is manual	Intake → project → channel is automatic
P5	No record of who decided what	Decisions happen in heads and DMs	An audit log of every action with its reason (D9)

4. Scope

In scope: everything in the brief: Gmail monitoring, parsing, validation with clarification, voice tier confirmation, the Rocketlane project with the right template and timeline (created, extended with a deal-specific task, and verified through the project, phase and task APIs), the personalised Slack channel, the overdue-escalation automation, guardrails, audit logging and tests.

Out of scope, with a reason:

Item	Why not now
Scheduling the kickoff call	Needs the customer's availability; it is the CSM's first template task
Handoff summary to Support	Weeks later; a next step (§13)

Item	Why not now
Escalating *blocked* tasks (Priya's "> 3 days")	The brief specifies *overdue* +1/+4, built as specified. "Blocked" is a different signal; it is one more Rocketlane rule, offered in §16 and not built until confirmed
Reading the tier from Salesforce	The brief requires the call; §13 explains why Salesforce should be the source in production

5. Assumptions and open questions

Questions were emailed to Karthik on 27 Oct. Until answered, the assumption is what the system does; each is a config value or one function.

ID	Question	Assumption used
A1	Where does the AE's phone number come from?	An **AE directory** (email → name, phone) in `config/team.yaml`. It is also the sender allow-list
A2	What identifies a duplicate?	The **Salesforce opportunity ID** from the link, normalised to its 18-character form, stored on the project as `externalReferenceId`
A3	Who are the Project Manager and the Project Owner?	**Owner = the CS Manager** (Rocketlane's project owner). **PM = the assigned CSM**, through a "Project Manager" placeholder in both templates
A4	Who is told when the system cannot decide?	The **CS Manager**, in Slack `#cs-onboarding-escalations`
A5	When does the timeline start?	The **next business day** after the email. The length comes from the template (30 or 14 days)
A6	Required fields	Customer company name, customer contact email, Salesforce opportunity link, from a known AE. A signature naming a different AE is asked about (whom to call?). Contact name, close date and notes are optional
A7	Which CSM gets an Enterprise deal?	The next dedicated CSM in `team.yaml`. Round-robin by workload is a production step
A8	"Shared Slack channel with the customer"	Production uses **Slack Connect** (paid). The build creates the channel in our workspace; the one differing call is shown. `SLACK_MODE=console` simulates Slack in the terminal if no workspace is available
A9	Voice calls to Indian numbers on free tiers	Checked 28 Oct: free Vapi numbers cannot dial abroad. The demo uses `VOICE_MODE=web`: the same assistant in the browser, same transcript and decision code, stated in the video. Production: a paid number (or Twilio) dials the AE
A10	Rocketlane trial (business email only)	Granted 28 Oct; every adapter run against the live trial (§14). The fake mirrors what it returned
A11	Does the customer get a Rocketlane invite on day one?	Yes: they own the data sign-off task, so they are added as a customer user and Rocketlane invites them

6. The proposed agentic workflow (to-be)

```
 1  New email in the CS inbox: any message without the "novacrm/processed" label
    (read or unread does not matter: the inbox is shared); the agent's own replies are skipped
 2  Intake: is the sender a known AE?                                  no -> REJECTED
 3  Intake: LLM extracts fields; code validates and grounds them   missing -> ask in thread
 4  Intake: already in Rocketlane?  same opportunity -> DUPLICATE, AE told
                                    same customer, made by hand -> a person decides
 5  Intake: phone the AE: "Enterprise or Growth?", then a read-back
        confirmed -> 6       unsure -> a person       no answer / ambiguous -> call again
        (at most 3 calls per deal; an ambiguous answer gets one more call, not two)
 6  Intake: tier -> template, CSM, start date (code)
 7  Intake: create the project from the template; add the deal's kickoff task (Rocketlane API)
 8  Intake: verify template, length, phases, tasks, sign-off owner, CSM     no -> a person
    -- handoff: ProjectReady {deal, tier, csm, contact, project link, start} --
 9  Communication: create the channel, set the topic, invite, post the welcome
10  Done. The Rocketlane automation watches for overdue tasks from here.
    Every step writes an audit record: time, agent, action, inputs, outputs, decision, reason.
```

6.1 States of one deal

Each deal has a short key (`D1`, `D2`, …) and is linked to its opportunity ID once that is known. Its state is saved before each next step, so a crash resumes where it stopped.

State	Meaning	Next
`RECEIVED`	Email picked up, or the AE answered a question	`REJECTED`, `NEEDS_INFO`, `DUPLICATE`, `AWAITING_TIER`, `HUMAN_REVIEW`
`REJECTED`	Unknown sender, or not a deal email (no deal field found at all)	(end)
`NEEDS_INFO`	The AE was asked for missing fields (at most 2 asks)	`RECEIVED` on the AE's reply; `HUMAN_REVIEW` after 24 h or a third incomplete answer
`DUPLICATE`	A project for this opportunity already exists; the AE got its link	(end)
`AWAITING_TIER`	Calling the AE; call ID, attempts and next call time are saved	`TIER_CONFIRMED`, `HUMAN_REVIEW`
`TIER_CONFIRMED`	Tier confirmed by the AE, or set by a person	`PROJECT_CREATED`, `HUMAN_REVIEW`
`PROJECT_CREATED`	Created and verified; the handoff is sent	`CHANNEL_CREATED`, `HUMAN_REVIEW`
`CHANNEL_CREATED`	Done	(end)
`HUMAN_REVIEW`	A person must act; the deal records **why** and **where to resume**	see below
`CLOSED`	Closed by a person, or merged into another deal for the same opportunity	(end)

An outage is not a state. If Rocketlane or Slack is down, the deal keeps its state and the same step is retried on later polls (at most every 5 minutes). The CS Manager is told once after 1 hour ("will resume on its own, no action needed"); after 24 hours a person decides.

New messages in a thread move a deal only when the message is from the deal's AE and the deal has no complete email yet (waiting for missing fields, or with a person about them), or was judged "not a deal" (so a real deal sent into that thread later is not lost). Anything else (a "thanks", a colleague's reply, a message seen again after a crash) is logged as `followup_ignored` and changes nothing.

Leaving `HUMAN_REVIEW`, by reason (each exit is logged with the person's name):

Reason	Exit
AE unreachable, unsure, or ambiguous	`novacrm resolve D3 --tier growth --by priya` (allowed only for these reasons, and only when every required field is valid), or `retry` to call the AE again
Missing fields not supplied	The AE replies in the thread, or `retry` (re-reads the thread), or `close`
The email could not be read (LLM down for 3 polls)	`retry` (re-reads the email)
A project for this customer exists that the agent did not make	`close --reason duplicate`, or `retry --not-duplicate` (continue; recorded)
Rocketlane or Slack rejected a request (4xx), or down for 24 h	Fix the cause, then `retry` (resumes at the saved step)
The created project does not match its template	Fix or delete the project in Rocketlane, then `retry` (re-verifies, or re-creates if deleted)
An unexpected error in one deal (a bug)	`retry` after the fix; the other deals keep moving meanwhile
Any	`novacrm close D3 --by priya --reason "..."`

7. Key decisions

D1. A fixed workflow with two LLM-backed agents, not an autonomous planner

The workflow is the same ten steps every time. An agent that plans its own tool calls adds risk (skipped checks, invented steps) and blurs the audit trail, for no gain. A small state machine in plain Python runs the steps. The system is multi-agent in the sense the brief means: two agents with separate responsibilities, each using an LLM where text must be understood or written, that meet at one typed, logged handoff (`ProjectReady`):

- **Intake & Routing Agent:** the LLM extracts fields and interprets the call transcript. Code validates, checks duplicates, retries, maps tier → template and calls the APIs.
- **Voice assistant** (on Vapi, part of the Intake Agent's step 5): a conversational agent with one job, to get an explicit "Enterprise" or "Growth" and read it back. It says it is automated.
- **Communication Agent:** the facts (tier, CSM, dates, link) are filled in by code. The LLM writes one opening line from the AE's notes under a hard rule: at most 25 words, no digits, no plan names, no names other than the customer's; otherwise a standard line is used.

Alternative: LangGraph or CrewAI. A framework pays off when the control flow is dynamic. Here it is not, and a short explicit state machine is easier for NovaCRM's team to read and change.

D2. Extraction is schema-bound and grounded: the model may not invent a field

The email thread is passed to the LLM inside delimiters, as data and not instructions, with a fixed JSON schema. Code then checks every value:

- **Well-formed:** a valid contact email; a Salesforce opportunity URL whose ID (`006…`, 15 or 18 characters) is normalised to its 18-character form, so both forms of one opportunity match.
- **Grounded:** each value must appear in the thread (ignoring case, spacing and punctuation). A value not found there counts as missing and is never used. This is "ask rather than guess" in code.

- **The AE** is the sender, from the directory. If the signature names a different AE, the agent asks whom to call.
- A known AE's email is rejected as "not a deal" only if no deal field is found at all, so a misjudged email gets a question instead of disappearing.

If anything is missing, the agent replies in the thread, listing exactly what it needs, and re-reads the thread when the AE answers. If the LLM cannot read an email for 3 polls, a person is asked.

D3. Tier confirmation: two signals must agree; "unsure" is not "ambiguous" or "unreachable"

The assistant asks: "Is Acme on the Enterprise plan or the Growth plan?" and reads the answer back: "Just to confirm: Enterprise, the 30-day onboarding with a dedicated CSM. Is that right?"

After the call:

1. **The LLM** reads the transcript: `{outcome: enterprise | growth | unsure | unclear, readback_confirmed, evidence_quote}`.
2. **Code** checks the transcript on its own: the AE named the tier, the assistant's read-back named the same tier, the AE answered it with a plain, unhedged yes ("Yeah, I guess so, probably", "Sure, but let me check with finance" and "Yes, unless they downgrade" are not; "Uh, yes", "That's correct" and "Absolutely" are), and did not name a different tier afterwards ("Yes. Oh sorry, Growth actually" is not a confirmation). The LLM's quote must be the AE's own words naming the tier. A test with a model that always says "growth" shows this check stands on its own.

The tier is `CONFIRMED` only when both agree. Otherwise:

Outcome	Example	What happens
Unreachable	No answer, busy, voicemail (or a greeting that never answers the question), call failed, or no result from the platform within 10 minutes	Call again 15 minutes later (seconds in the demo)
Unsure	"I'm not sure, check with finance"	A person, at once: calling again will not help
Ambiguous	"Enterprise… or maybe Growth"; read-back denied; the two signals disagree	One more call, then a person

One counter covers all of it: **at most 3 calls per deal**. The CS Manager gets the reason and the transcript. If the LLM is down when the call ends, the outcome is ambiguous, never a tier. **No path in the code assumes a tier.**

D4. Rocketlane: create from the template, add one task, then verify through the API

The two templates (§9) are built in Rocketlane, so CS Ops can change the plan without code. The tier policy in code is the only mapping:

Tier	Template	Length	CSM (= Project Manager)
Enterprise	`Enterprise Onboarding — 30 day`	30 days	A named, dedicated CSM
Growth	`Growth Onboarding — 14 day`	14 days	The pooled Growth CSM

The template creates the standard plan; the API creates the deal-specific part and checks the rest:

1. **Create:** `POST /api/1.0/projects` with `sources: [{templateId, startDate}]`, `startDate`, the owner (CS Manager), the customer company (`autoCreateCompany`), the CSM as a team member, the contact in `teamMembers.customers`, the "Project Manager" placeholder mapped to the CSM, the "Customer data owner" placeholder mapped to the contact, and `externalReferenceId` = opportunity ID. No `dueDate`: with a template, Rocketlane computes it. Rocketlane creates the 4 phases and 15 tasks from the template.
2. **Add the deal's task:** `POST /api/1.0/tasks`: "Kickoff call with John Doe" in the Kickoff phase, owned by the CSM, due on day 3 (Enterprise) or 2 (Growth), carrying the AE's notes from the email.
3. **Verify** (`GET /projects/{id}` with its sources, `GET /phases`, `GET /tasks`): the source template's **name** contains the tier (this catches a wrong template ID in config, which re-reading the ID we sent could not), the planned duration equals the value measured in the trial and stored in config, the 4 phases and 15 template tasks are there, the sign-off task is assigned to the customer (the contact, or the customer placeholder mapped to the contact; tasks are listed with `includeFields=phase,assignees`, since both are optional fields), and the CSM is on the project. A mismatch goes to a person.

At start-up the agent refuses a config in which two tiers share a template ID. (The API has no template endpoint, so the template's name can only be checked on a created project.)

Fallback, built and switchable: `rocketlane.mode: api` creates every phase (`POST /phases`), task (`POST /tasks`) and the go-live dependency (`POST /tasks/{id}/add-dependencies`) from `config/onboarding_plan.yaml`, each write preceded by a lookup by name so a retry never creates twice. Templates stay the default because CS Ops can then change the plan without code; the fallback exists because the trial may not apply templates through the API, and in it the escalation automation is set globally instead of on the templates.

D5. Writes that must not happen twice

Five external writes are not safe to repeat. For each, the deal records what it intends before the write and the result right after it, so a resume can tell whether it already happened. No clocks from different machines are compared.

Write	Saved before	On resume or retry
Phone call	attempt number and the next allowed call time	Call ID saved → fetch its result. No ID (a crash in the second between placing and saving) → the attempt still counts and the spacing still applies: at most one extra call, never back-to-back
Create project	the step-4 result: no project had this opportunity's reference	Project ID saved → use it. Otherwise look up the reference **before every retry**: since none existed at step 4 and a person making a project by hand does not set it, a project carrying it is ours (adopt). None → create
Create kickoff task	the project ID	Look for the task by name in the project; adopt it if found
Create channel	the intended name; the channel is marked with the opportunity	Channel ID saved → use it. Name taken → adopt it only if our bot made it for this opportunity and no other deal uses it; otherwise the next suffix (`onb-acme-corp-2`)
Post welcome	the channel ID; the welcome is marked with the opportunity	Timestamp saved → skip. Otherwise skip only if this opportunity's welcome is already in the channel

Lookups and reads are retried freely; writes are never retried blindly. Setting the topic and inviting members are safe to repeat.

One opportunity, one deal. Step 4 looks up the opportunity's reference *and* the customer's name before anyone is called, so a project made by hand (which has no reference) is caught and a person decides. Several emails for one deal are merged, and the complete one always wins: - a new email for an opportunity that is already being handled (past reading its email) is merged into that deal and starts nothing; - an earlier deal still without a complete email (waiting for the AE, or already with a person about it) is merged into the new complete one when it has the same opportunity link (saved even from an incomplete email) or comes from the same AE with the same customer name. A complete resend is never held back by an incomplete first email, and the first one never pages Priya.

D6. External failures

Failure	Behaviour
Timeout, 5xx or 429 on a read	Up to 3 quick retries (1, 2, 4 s with jitter)
Timeout or 5xx on a write	Outcome unknown: the D5 rule decides on the next attempt. 429 means "not done", so it is retried the same way
Still failing (at any step, including the duplicate check before the call)	The deal keeps its state and waits; retried on each poll at most every 5 minutes; Priya told once at 1 h; a person after 24 h. No call is made while the duplicate check cannot run
4xx (bad request, bad key)	Not retried; a person, with the error
The voice platform fails to place the call, or never reports the call as ended (10 min)	A failed attempt, under the call counter
A bug in one deal's step	That deal goes to a person (`internal_error`); every other deal keeps moving
LLM unavailable or invalid JSON	One retry; the email waits for the next poll; after 3 polls, a person

D7. The inbox trigger

Every 30 seconds the agent asks Gmail for inbox messages from the last 7 days **without** the `novacrm/processed` label, excluding mail the inbox sent itself. It does **not** use read/unread: the CS inbox is shared, and a person opening an email first must not hide the deal. A message is labelled only after its deal is saved, and a message ID seen before is skipped, so a crash between the two cannot process it twice. Polling needs no public URL; production uses Gmail push (§13).

D8. The Communication Agent

- **Channel name:** `onb-<customer>`, within Slack's rules (lowercase, digits, hyphens, ≤ 80 characters), e.g. `onb-acme-corp`.
- **Topic:** plan, onboarding window and target go-live date, CSM, and the Rocketlane project link.
- **Welcome message:** one template per tier. Enterprise introduces the named dedicated CSM, the 30-day plan by phase and a weekly sync. Growth introduces the CS team, the 14-day plan and how to get help. Both list the customer's first three steps. The opening line is from D1.
- **Members:** the CSM and the AE. In production the customer is invited through Slack Connect (`conversations.inviteShared`); the build shows that call as the production difference.

D9. The audit log

Every action is one row in the `audit` table: time, deal, agent, action, inputs, outputs, decision and reason. Phone numbers and customer emails are masked (`******1234`, `j***@acme.com`). `novacrm trace <deal>` prints a deal's timeline; the video uses it to show each guardrail firing.

8. Guardrails

Requirement or risk	Guardrail	Tests
Missing data must not be guessed (brief)	Schema-bound extraction + grounding; ask in the thread; nothing created	`test_validation.py`
No project without a clear tier (brief)	Two signals + read-back; retry or escalate by outcome; no default tier exists; `resolve` only for tier reasons with valid fields	`test_voice.py`
Every action logged with its reason (brief)	An audit row per action: time, agent, action, inputs, outputs, decision, reason	`test_happy_path.py` checks the whole trail
Template mix-up	One tier policy; a start-up config check; the created project verified by template name, length, phases, tasks and people	`test_rocketlane.py`
Duplicate projects, tasks, channels, messages, calls	Intent before every unsafe write, a lookup before every retry (D5); IDs normalised; emails merged	`test_rocketlane.py`, `test_slack_and_guardrails.py`
Stray or spoofed email	Only directory senders; only the deal's AE can move it; production adds Gmail's DKIM/SPF result	`test_validation.py`
Prompt injection in an email	The email is data in delimiters; the model has no tools; output is schema-checked and grounded; the tier comes only from the call	`test_instructions_inside_an_email_cannot_skip_the_call`
Wrong facts in the welcome message	Facts from code; the LLM's line under a hard rule	`test_the_welcome_line_cannot_state_facts_…`
Calling the AE too often	At most 3 calls, spaced; "unsure" stops the calls	`test_voice.py`
PII in logs	Phone numbers and contact emails masked; secrets only in `.env`	`test_the_audit_log_masks_…`

9. The onboarding templates (built in Rocketlane)

15 tasks across the 4 phases, in both templates. Days are offsets from the start; Growth compresses the same plan into 14 days. "PM" is the Project Manager placeholder (the CSM); "Customer" is the Customer data owner placeholder (the contact).

#	Phase	Task	Ent. (30 d)	Growth (14 d)	Owner
1	Kickoff	Internal handoff review with the AE (scope, success criteria, risks)	0–1	0	PM
2	Kickoff	Schedule and run the kickoff call	1–3	1–2	PM
3	Kickoff	Agree success criteria and the go-live date	3–5	2	PM
4	Kickoff	Share the onboarding plan; set up customer admin access	3–5	2	PM
5	Data Migration	Collect data exports and agree the field mapping	5–9	3–4	Customer
6	Data Migration	Test import into a sandbox	9–12	5	PM
7	Data Migration	Full data import	12–15	6	PM
8	Data Migration	**Customer data verification sign-off**: record counts and a sample check, confirmed by the customer	15–17	7	**Customer**
9	Configuration	Configure pipelines, fields and user roles	15–20	6–8	PM
10	Configuration	Integrations (Enterprise: email, calendar, SSO; Growth: email, calendar)	18–22	8–9	PM
11	Configuration	Training (Enterprise: live sessions; Growth: one group session + recordings)	20–25	9–10	PM
12	Go-Live	User acceptance testing with the customer's champions	24–27	10–11	Customer
13	Go-Live	Go-live readiness check (**depends on task 8**)	27–28	12	PM
14	Go-Live	Go-live and hypercare	28–30	12–14	PM
15	Go-Live	Handoff summary and transfer to Support	30	14	PM

This is the fix for pain P3: the data sign-off is owned by the customer, not the CSM, and go-live readiness depends on it, in the template, so no project can be created without it.

10. The overdue-escalation automation (configured in Rocketlane)

Rule	Trigger and condition	Action
1	A task is overdue, for **1 day**	Notify the **Project Manager** (the PM placeholder, i.e. the CSM)
2	A task is overdue, for **4 days**	Notify the **Project Owner** (the CS Manager)

Both rules are set on the two templates, so every project the agent creates gets them. If "Notify" cannot target a placeholder, rule 1 notifies the task's assignee (the CSM owns every PM task), and the video says so. For the recording, a test project is created early with tasks whose due dates pass 1 and 4 days before recording day, so the rules fire on real overdue tasks; the rule screens and run history are the fallback shot.

11. Test plan

220 tests run offline in under a second (`make test`), against in-memory fakes of Gmail, the voice platform, Rocketlane, Slack and the LLM. The fakes behave like the real services where it matters: Rocketlane builds

phases and tasks from the template, computes durations and does not enforce unique references, and can lose a response after a write has landed; Slack refuses a taken name. An opt-in `make smoke` runs the happy path against the real accounts.

Brief requirement	**Tests (`tests/`)**
Happy path	Enterprise and Growth emails → parse → call confirms → project from the right template with the right CSM, the contact on the sign-off task, and the kickoff task → personalised channel. The audit trail is checked step by step, including the `ProjectReady` handoff
Validation	Missing customer name; missing contact email; malformed email; a contact without an email; missing or non-Salesforce link; a signature naming another AE; a value the model invented; unknown sender; not a deal email; the AE's reply completes the deal; a third incomplete answer → person; no reply in 24 h → person; a stranger's reply in the thread changes nothing
Template accuracy	Enterprise → 30-day template, dedicated CSM; Growth → 14-day template, pooled CSM. A wrong length → person, then fixed and retried. Template IDs swapped in config → caught by the template name. Two tiers on one template → refused at start-up
Rocketlane down	A brief read failure retried in the step. A lost create response → exactly one project. Down after the call → waits, Priya told once at 1 h, resumes with no second call. Down before the call → no call until the duplicate check can run. Down for a day → person. A 4xx → person, not retried
Existing project	Same opportunity → `DUPLICATE`, no call. Made by hand for the customer → person before any call; `retry --not-duplicate` or `close`. Made by hand during the calls → not adopted. The 15-character form of the ID → recognised. A resend while calling → merged, no extra call. Incomplete email then a complete one in a new thread → proceeds once
AE does not answer	No answer ×3 → person after exactly 3 calls; calls are spaced; voicemail; a voice-API error; a later call succeeding
Ambiguous tier	Hedging between tiers → one more call → person. The two signals disagreeing. A correction after the read-back. The LLM down when the call ends. "I'm not sure" → person with no more calls. `resolve` and `retry` after review; the wrong exit refused
Edge cases	A model that always agrees cannot confirm alone (hedged yes, a read-back of a tier never said, a bare yes); a voicemail greeting; a call with no result; a crash after placing a call and after creating the kickoff task; a channel our bot made for another opportunity not reused; Slack's own messages not mistaken for the welcome; an ignored Rocketlane filter; a real deal sent into a "not a deal" thread; the first email missing the customer name; one broken deal not stopping the others
Also	Slack down → resumes with one channel; a lost channel-create response → adopted; someone else's channel name → suffix; a lost post response → no second welcome; channel-name rules; the welcome-line rule; prompt injection; PII masking; the 15 → 18 ID conversion

The real adapters have request-shape tests against a mock HTTP transport: the exact Rocketlane payloads and filters, paging, error mapping (429/5xx retry, 4xx not), and Vapi's `endedReason` codes.

12. Demo script (the video)

Positive run (about 4 minutes): send an Enterprise deal email → the agent picks it up and shows the extracted fields → the AE's phone rings; the AE answers "Enterprise" and confirms the read-back → the Rocketlane project appears (Enterprise template, CSM assigned, the contact on the sign-off task, the kickoff task with the AE's notes) → the Slack channel appears with its topic and welcome → the automation rules and their run history on the test project → `novacrm trace D1`.

Negative run (about 5 minutes): 1. No contact email → the agent asks in the thread → the AE replies → the deal continues. 2. A malformed opportunity link → the agent asks; nothing created. 3. The AE does not pick up → three calls (seconds apart for the demo) → message in Slack. 4. **Ambiguous:** "Enterprise… or actually maybe Growth", read-back denied → one more call → person. 5. **Unsure:** "I'm not sure, check with finance" → a person at once → Priya runs `novacrm resolve`. 6. A project made by hand for the customer → a person, before any call. 7. Rocketlane made unreachable after the call (`novacrm fault rocketlane down`) → the deal waits → `fault rocketlane up` → resumes, with no second call or project. 8. `novacrm trace` for each.

If Slack has to be faked, the video says so and shows what changes in production.

13. What changes in production

In the build	In production
Gmail polling	Gmail `users.watch` + Pub/Sub push
SQLite	Postgres; audit shipped to the logging stack
One process polling for call results	A worker queue; the voice step waits on the call-ended webhook
Channel in our workspace	Slack Connect invite to the customer
`.env` secrets	A secrets manager
AE directory in YAML	Salesforce users (the opportunity owner)
Calls at any hour	Calls only in the AE's working hours
Sender check by address	Plus Gmail's DKIM/SPF result
Tier confirmed by phone	**The tier is a field on the Salesforce opportunity.** Read it there; call only when it is empty or disagrees. That removes a call from almost every deal. The call is built because the brief requires it, and it stays the right fallback

Next steps: kickoff scheduling, a handoff summary drafted from the Rocketlane project, a weekly report of escalations by phase.

14. Platform checks (day 1 in the trial)

What the design depends on that the documentation does not settle. `novacrm verify-platform` runs the API checks automatically against the trial (one test project per tier, read back, then archived), saves the raw responses and prints a pass/fail table; the UI-only checks are listed for a person. `novacrm test-call` checks the voice route. Results are written here, and each has a fallback already built (D4 `api` mode, §10, A9 `web` mode) if the trial contradicts it.

Run on 28 Oct against a Rocketlane trial (Enterprise plan), Vapi, Slack, Gmail and Gemini. `verify-platform`: **17 of 17 checks pass**, both tiers.

Check	Result (28 Oct)
A template can be applied through the API (`sources` on create)	**Pass**: `sources` names the template on read-back
Planned duration of each template, as Rocketlane reports it	**Enterprise 29, Growth 13** (`plannedDurationInDays`). Rocketlane counts working days, so the calendar length moves with the start weekday (29 from a Wednesday, 31 from a Thursday). Verification allows ±3 days (a wrong template is off by ~16), and Slack states Rocketlane's own due date
Customer placeholder mapped at create	**Pass**: the sign-off task comes back assigned to the contact (`members`), not the placeholder
Placeholder IDs of both templates	Read from the API: Project Manager and Customer are account-wide placeholders, the same on both templates
A template task assigned to the customer comes back under `assignees`	**Pass** (as `members` after mapping)
The task-8 → task-13 dependency survives the template	**Pass**: read back on both tiers. Whether it *blocks* starting task 13 is a UI check
Archived projects appear in the `externalReferenceId` lookup	**Pass**, but only after a short delay; the check re-reads
"Notify" can target the Project Manager placeholder	**Pass**: offered as a recipient. The rule is one automation per task ("When task becomes overdue", condition "overdue by N days") with two blocks: 1 day → Project Manager, 4 days → Project owner; saved once and applied to all 15 tasks of both templates
A task with a past due date fires the rules	Test project created 10 days back-dated (tasks 1 and 8–9 days overdue); see the video
Vapi can call a +91 number	**No**: "Free Vapi numbers do not support international calls". `VOICE_MODE=web` (same assistant, browser audio) is used for the demo; production uses a paid number

Found by the live runs, and handled in the code:

- **Creating from a template takes ~27 s**, and once took over 20 s while the project *was* created. The read timeout is now 120 s, and the existing D5 path adopts a project whose create response was lost (looked up by `externalReferenceId` before any retry).
- **Template tasks are copied in the background**: a read right after create can show the four phases and no tasks. The verify step now re-reads after 5, 10 and 15 s before calling a short task list a mismatch (two tests).
- **Rocketlane rejects a duplicate** `externalReferenceId` ("Invalid External Reference Key"): a second guard against duplicate projects, behind ours. A rejected create (400) can still leave the project behind, so a person resolving it looks before retrying.
- **A customer user belongs to one company**: adding an existing contact of another company is a 400. The agent sends it to a person (`rejected_by_rocketlane`).
- **Speech recognition**: one live answer "Enterprise" came back as "Interface name". The decision code did not guess; the assistant now boosts the two plan words (Deepgram key terms) and asks again on an unclear answer instead of ending the call.
- Slack message metadata round-trips (`novacrm_welcome`), so the posted-once guard works on the real API.

---

15. Risks and plan B

Risk	Plan
No Rocketlane trial (business email only)	Resolved 28 Oct: trial account granted; all checks run live (§14)
The trial lacks the API or automations	Ask for a sandbox; the D4 fallback covers templates
The free voice tier cannot call +91	Confirmed on 28 Oct. The demo uses the browser call (same assistant), stated openly; production uses a paid number
Speech recognition mishears the tier	The read-back, and two-signal agreement
The LLM extracts a plausible but wrong value	Grounding: the value must be in the email
A customer name typed differently creates a second Rocketlane company	Accepted for now; production matches against existing companies first

16. What I would ask Priya next

1. Who owns the plan tier in Salesforce today, and how often is it wrong or empty? (It decides whether the call stays a fallback.)
2. Should a task that is *blocked* for more than 3 days escalate too, not only an overdue one?
3. What does "verified data" mean for your customers: record counts, a sample check, or a named person's sign-off?

Appendix. Code layout and stack

Each external system sits behind a small interface (`Inbox`, `VoiceCaller`, `ProjectStore`, `ChatWorkspace`, `LLM`) with a real adapter and an in-memory fake; the agents see only the interfaces, which is what makes the failure cases reproducible in tests.

```
src/novacrm/
  domain.py            tiers, tier policy, the plan constants, the deal record
  plan.py              the onboarding plan (config/onboarding_plan.yaml), dated per tier
  orchestrator.py      the state machine, and the human exits (resolve, retry, close)
  agents/base.py       move, wait out an outage, escalate: every change saved and audited
  agents/intake.py     screening, merging, duplicates, calls, project, plan, verification
  agents/extraction.py the email: schema-bound extraction, grounding, the clarification reply
  agents/tier.py       the call: transcript reading + the independent code check
  agents/comms.py      the Communication Agent: channel, topic, welcome
  adapters/            gmail vapi webcall rocketlane slack console gemini http
  fakes.py, sandbox.py in-memory versions of every adapter; the sample world for tests and demo
  faults.py            fault switches for the live demo (`novacrm fault rocketlane down`)
  verify.py            the day-1 platform checks (`novacrm verify-platform`)
  demo.py              every scenario offline (`novacrm demo`)
  store.py             SQLite: deals, threads, messages, the audit table (masked)
  cli.py               run, deals, trace, resolve, retry, close, check, test-call, demo, fault
config/                team.example.yaml, onboarding_plan.yaml, vapi_assistant.json, slack
manifest
docs/                  this document, SETUP.md, ROCKETLANE_SETUP.md, API_REQUESTS.md
(generated)
tests/                 one file per brief requirement, edge cases, api mode, adapter requests
```

Python 3.11 with `uv`, Pydantic, `httpx`, the Google API client for Gmail, Gemini Flash-Lite behind the `LLM` interface, SQLite and `pytest`.